AWS CICD

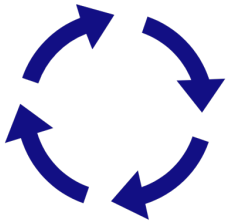
CodeCommit, CodePipeline, CodeBuild, CodeDeploy, …

CICD – Introduction

- We have learned how to:
  - Create AWS resources, manually (fundamentals)
  - Interact with AWS programmatically (AWS CLI)
  - Deploy code to AWS using Elastic Beanstalk
- All these manual steps make it very likely for us to do mistakes!

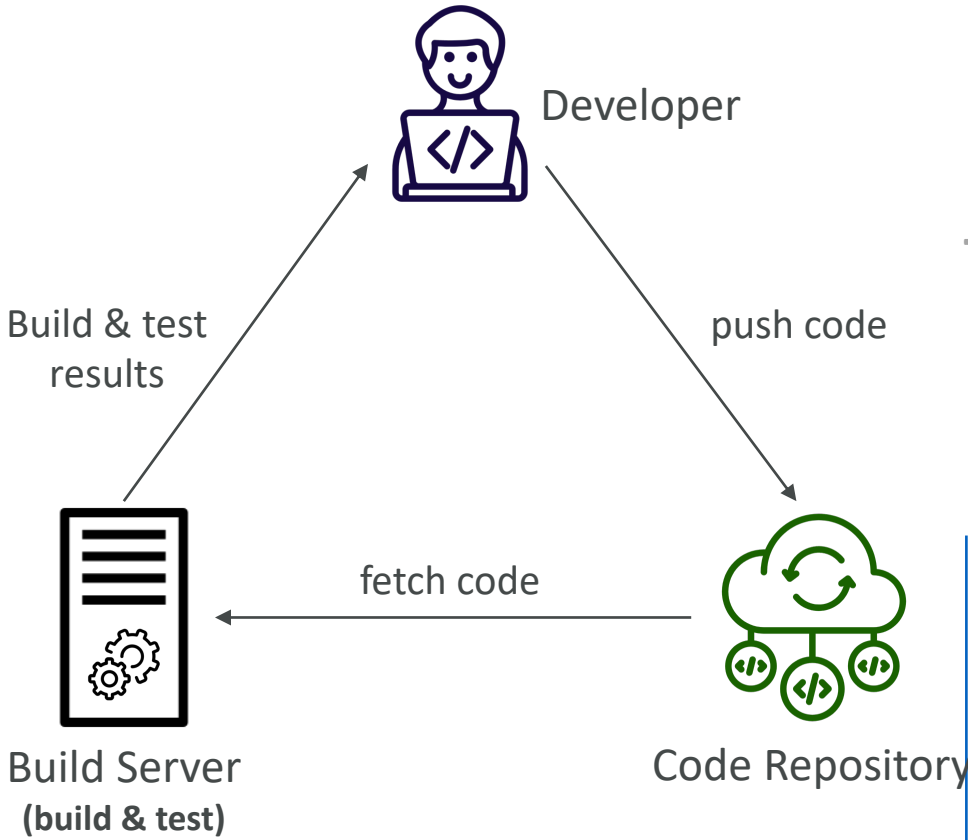
- We would like our code "in a repository" and have it deployed onto AWS
  - Automatically
  - The right way
  - Making sure it's tested before being deployed
  - With possibility to go into different stages (dev, test, staging, prod)
  - With manual approval where needed
- To be a proper AWS developer… we need to learn AWS CICD

Continuous Integration (CI)



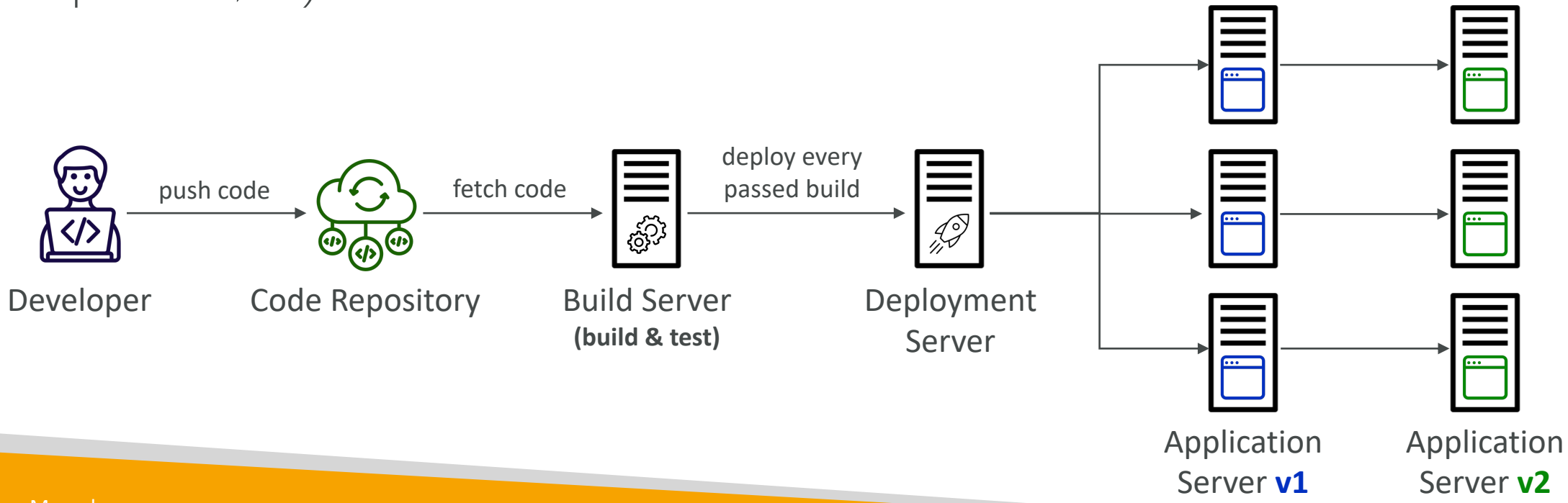
- Developers push the code to a code repository often (e.g., GitHub, CodeCommit, Bitbucket…)
- A testing / build server checks the code as soon as it's pushed (CodeBuild, Jenkins CI, …)
- The developer gets feedback about the tests and checks that have passed / failed

- Find bugs early, then fix bugs
- Deliver faster as the code is tested
- Deploy often
- Happier developers, as they're unblocked

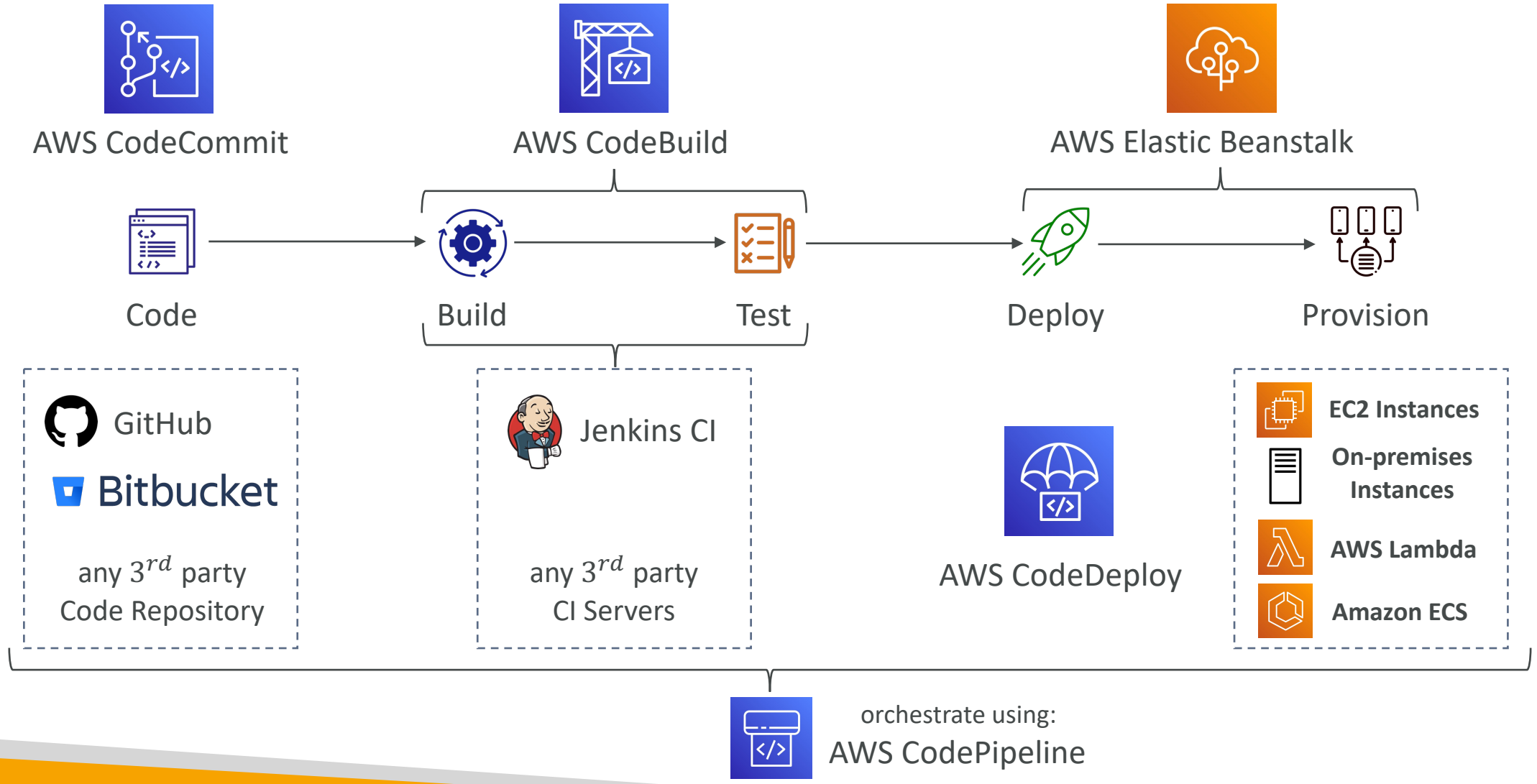


Continuous Delivery (CD)

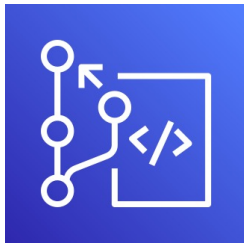
- Ensures that the software can be released reliably whenever needed
- Ensures deployments happen often and are quick
- Shift away from "one release every 3 months" to "5 releases a day"
- That usually means automated deployment (e.g., CodeDeploy, Jenkins CD, Spinnaker, …)



Technology Stack for CICD



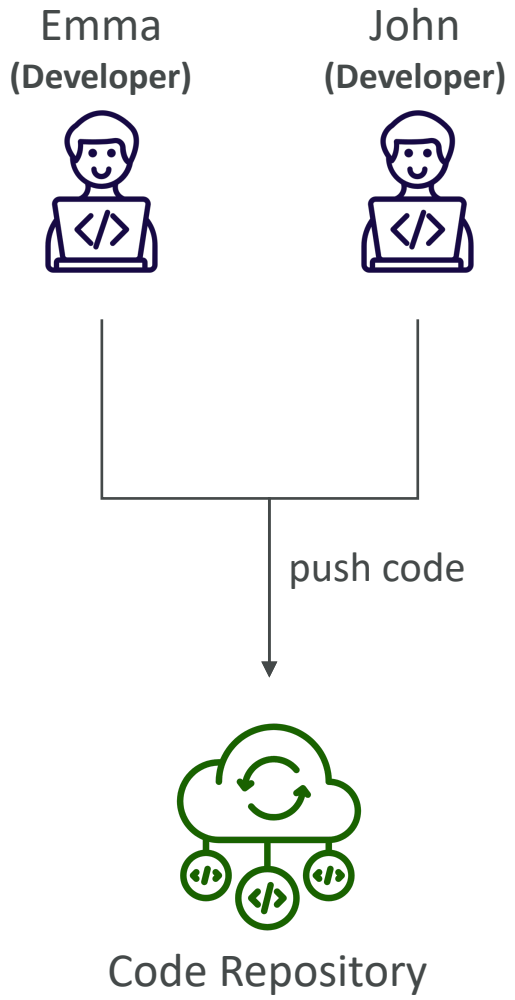
AWS CodeCommit



- **Version control** is the ability to understand the various changes that happened to the code over time (and possibly roll back)
- All these are enabled by using a version control system such as **Git**
- A Git repository can be synchronized on your computer, but it usually is uploaded on a central online repository
- Benefits are:
  - Collaborate with other developers
  - Make sure the code is backed-up somewhere
  - Make sure it's fully viewable and auditable

AWS CodeCommit

- Git repositories can be expensive
- The industry includes GitHub, GitLab, Bitbucket, …
- And **AWS CodeCommit:**
  - Private Git repositories
  - No size limit on repositories (scale seamlessly)
  - Fully managed, highly available
  - Code only in AWS Cloud account => increased security and compliance
  - Security (encrypted, access control, …)
  - Integrated with Jenkins, AWS CodeBuild, and other CI tools



CodeCommit – Security

- Interactions are done using Git (standard)
- **Authentication**
  - **SSH Keys** – AWS Users can configure SSH keys in their IAM Console
  - **HTTPS** – with AWS CLI Credential helper or Git Credentials for IAM user
- **Authorization**
  - IAM policies to manage users/roles permissions to repositories
- **Encryption**
  - Repositories are automatically encrypted at rest using AWS KMS
  - Encrypted in transit (can only use HTTPS or SSH – both secure)
- **Cross-account Access**
  - Do NOT share your SSH keys or your AWS credentials
  - Use an IAM Role in your AWS account and use AWS STS (**AssumeRole** API)

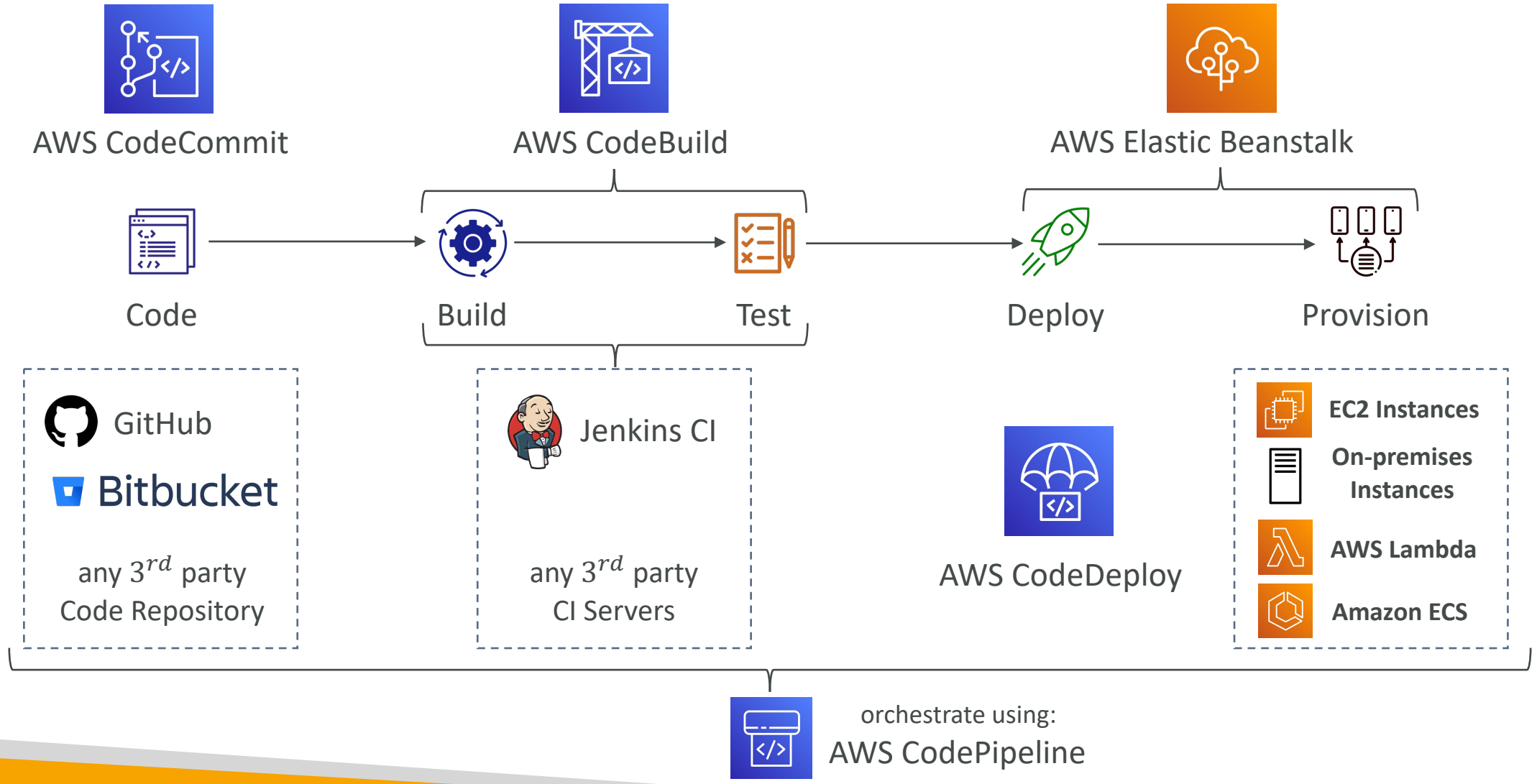
CodeCommit vs. GitHub

	CodeCommit	GitHub
Support Code Review (Pull Requests)	✔	✔
Integration with AWS CodeBuild	✔	✔
Authentication (SSH & HTTPS)	✔	✔
Security	IAM Users & Roles	GitHub Users
Hosting	Managed & hosted by AWS	- Hosted by GitHub - GitHub Enterprise: self hosted on your servers
UI	Minimal	Fully Featured

AWS CodePipeline

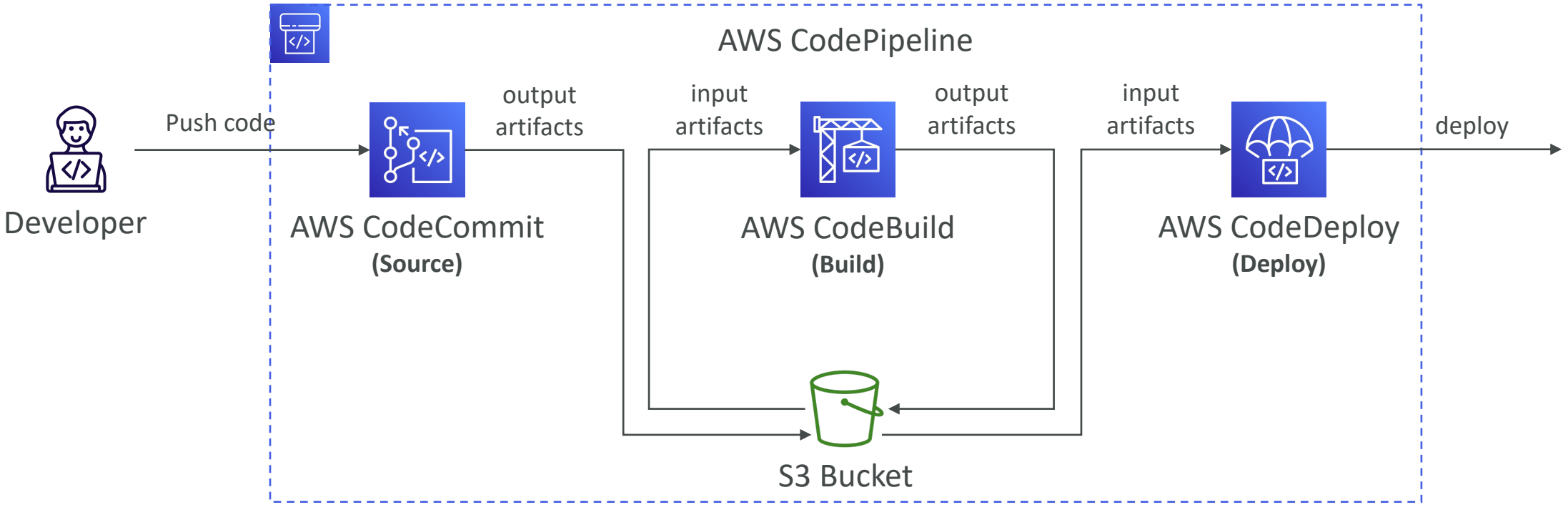
- Visual Workflow to orchestrate your CICD
- **Source** – CodeCommit, ECR, S3, Bitbucket, GitHub
- **Build** – CodeBuild, Jenkins, CloudBees, TeamCity
- **Test** – CodeBuild, AWS Device Farm, 3rd party tools, …
- **Deploy** – CodeDeploy, Elastic Beanstalk, CloudFormation, ECS, S3, …
- **Invoke** – Lambda, Step Functions
- Consists of stages:
  - Each stage can have sequential actions and/or parallel actions
  - Example: Build ➔ Test ➔ Deploy ➔ Load Testing ➔ …
  - Manual approval can be defined at any stage

Technology Stack for CICD



CodePipeline – Artifacts

- Each pipeline stage can create **artifacts**
- Artifacts stored in an S3 bucket and passed on to the next stage



CodePipeline – Troubleshooting

- For CodePipeline Pipeline/Action/Stage Execution State Changes
- Use **CloudWatch Events (Amazon EventBridge).** Example:
  - You can create events for failed pipelines
  - You can create events for cancelled stages

- If CodePipeline fails a stage, your pipeline stops, and you can get information in the console
- If pipeline can't perform an action, make sure the "IAM Service Role" attached does have enough IAM permissions (IAM Policy)
- AWS CloudTrail can be used to audit AWS API calls

AWS CodeBuild

- A fully managed continuous integration (CI) service
- Continuous scaling (no servers to manage or provision – no build queue)
- Compile source code, run tests, produce software packages, …
- Alternative to other build tools (e.g., Jenkins)
- Charged per minute for compute resources (time it takes to complete the builds)
- Leverages Docker under the hood for reproducible builds
- Use prepackaged Docker images or create your own custom Docker image
- Security:
  - Integration with KMS for encryption of build artifacts
  - IAM for CodeBuild permissions, and VPC for network security
  - AWS CloudTrail for API calls logging

AWS CodeBuild

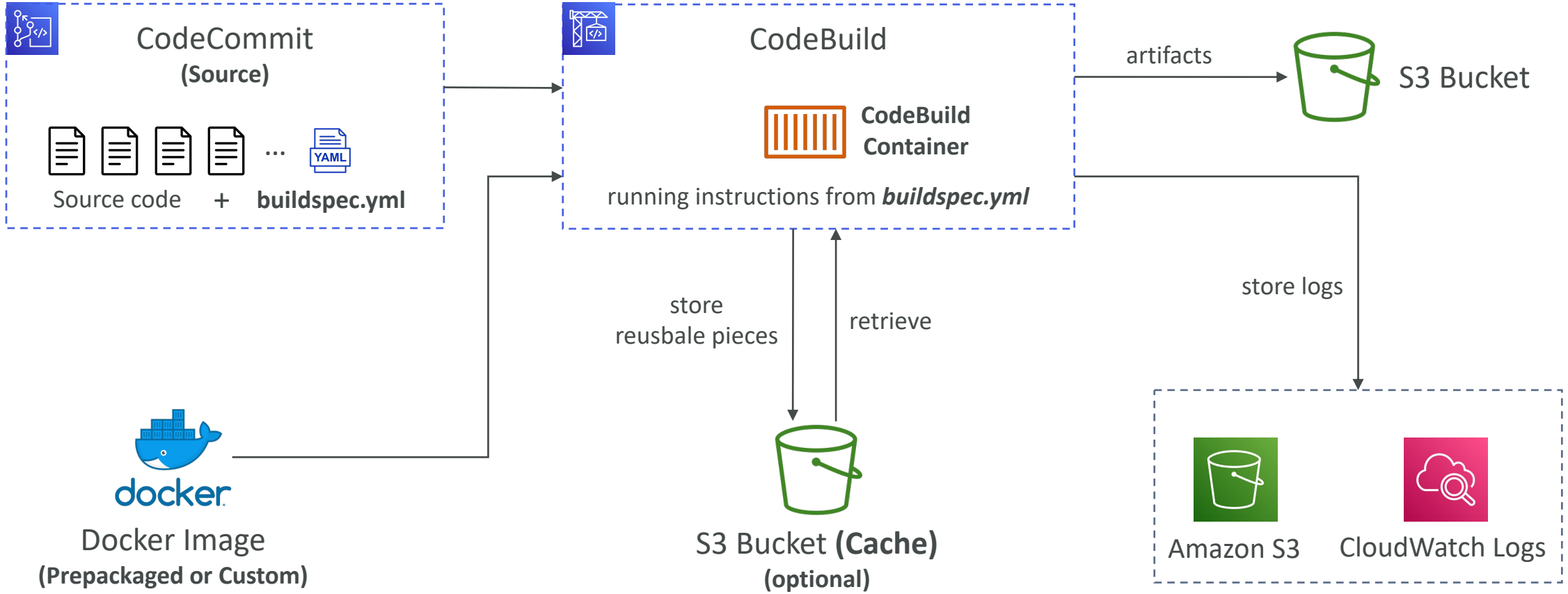
- **Source** – CodeCommit, S3, Bitbucket, GitHub
- **Build instructions**: Code file **buildspec.yml** or insert manually in Console
- **Output logs** can be stored in Amazon S3 & CloudWatch Logs
- Use CloudWatch Metrics to monitor build statistics
- Use EventBridge to detect failed builds and trigger notifications
- Use CloudWatch Alarms to notify if you need "thresholds" for failures

- **Build Projects can be defined within CodePipeline or CodeBuild**

CodeBuild – Supported Environments

- Java
- Ruby
- Python
- Go
- Node.js
- Android
- .NET Core
- PHP
- Docker – extend any environment you like

CodeBuild – How it Works



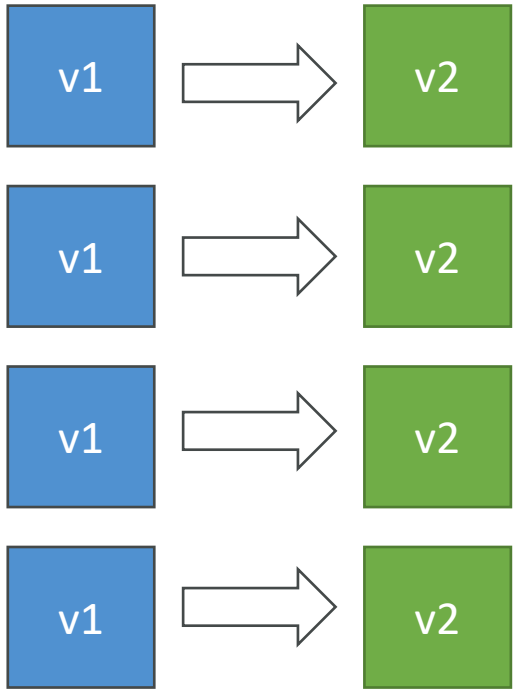
CodeBuild – buildspec.yml

- **buildspec.yml** file must be at the **root** of your code
- **env** – define environment variables
  - **variables** – plaintext variables
  - **parameter-store** – variables stored in SSM Parameter Store
  - **secrets-manager** – variables stored in AWS Secrets Manager
- **phases** – specify commands to run:
  - **install** – install dependencies you may need for your build
  - **pre_build** – final commands to execute before build
  - **Build** – **actual build commands**
  - **post_build** – finishing touches (e.g., zip output)
- **artifacts** – what to upload to S3 (encrypted with KMS)
- **cache** – files to cache (usually dependencies) to S3 for future build speedup

```yaml
version: 0.2

env:
  variables:
    JAVA_HOME: "/usr/lib/jvm/java-8-openjdk-amd64"
  parameter-store:
    LOGIN_PASSWORD: /CodeBuild/dockerLoginPassword

phases:
  install:
    commands:
      - echo "Entered the install phase..."
      - apt-get update -y
      - apt-get install -y maven
  pre_build:
    commands:
      - echo "Entered the pre_build phase..."
      - docker login -u User -p $LOGIN_PASSWORD
  build:
    commands:
      - echo "Entered the build phase..."
      - echo "Build started on `date`"
      - mvn install
  post_build:
    commands:
      - echo "Entered the post_build phase..."
      - echo "Build completed on `date`"

artifacts:
  files:
    - target/messageUtil-1.0.jar

cache:
  paths:
    - "/root/.m2/**/*"
```

AWS CodeDeploy

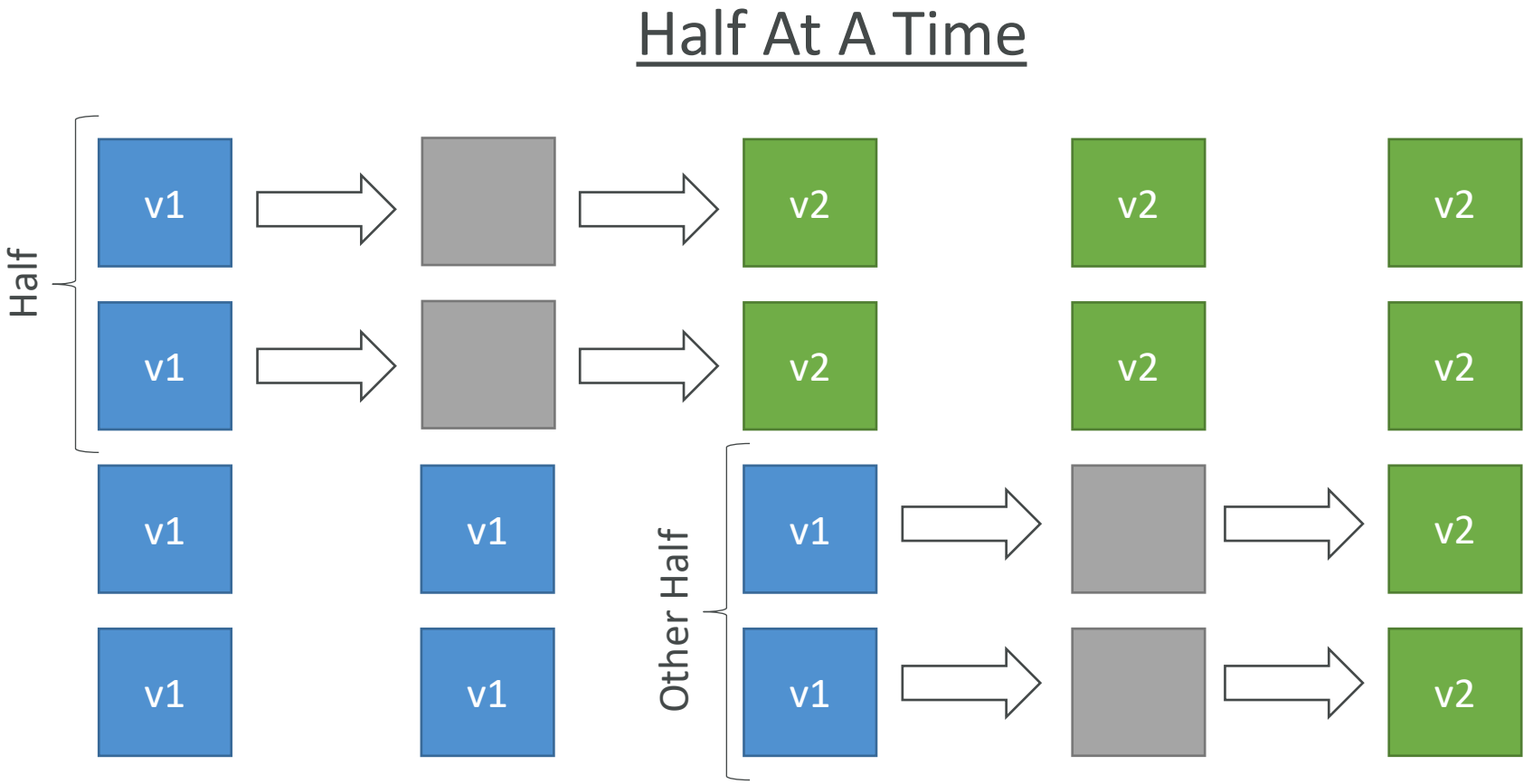
- Deployment service that automates application deployment
- Deploy new applications versions to EC2 Instances, On-premises servers, Lambda functions, ECS Services
- Automated Rollback capability in case of failed deployments, or trigger CloudWatch Alarm
- Gradual deployment control
- A file named appspec.yml defines how the deployment happens



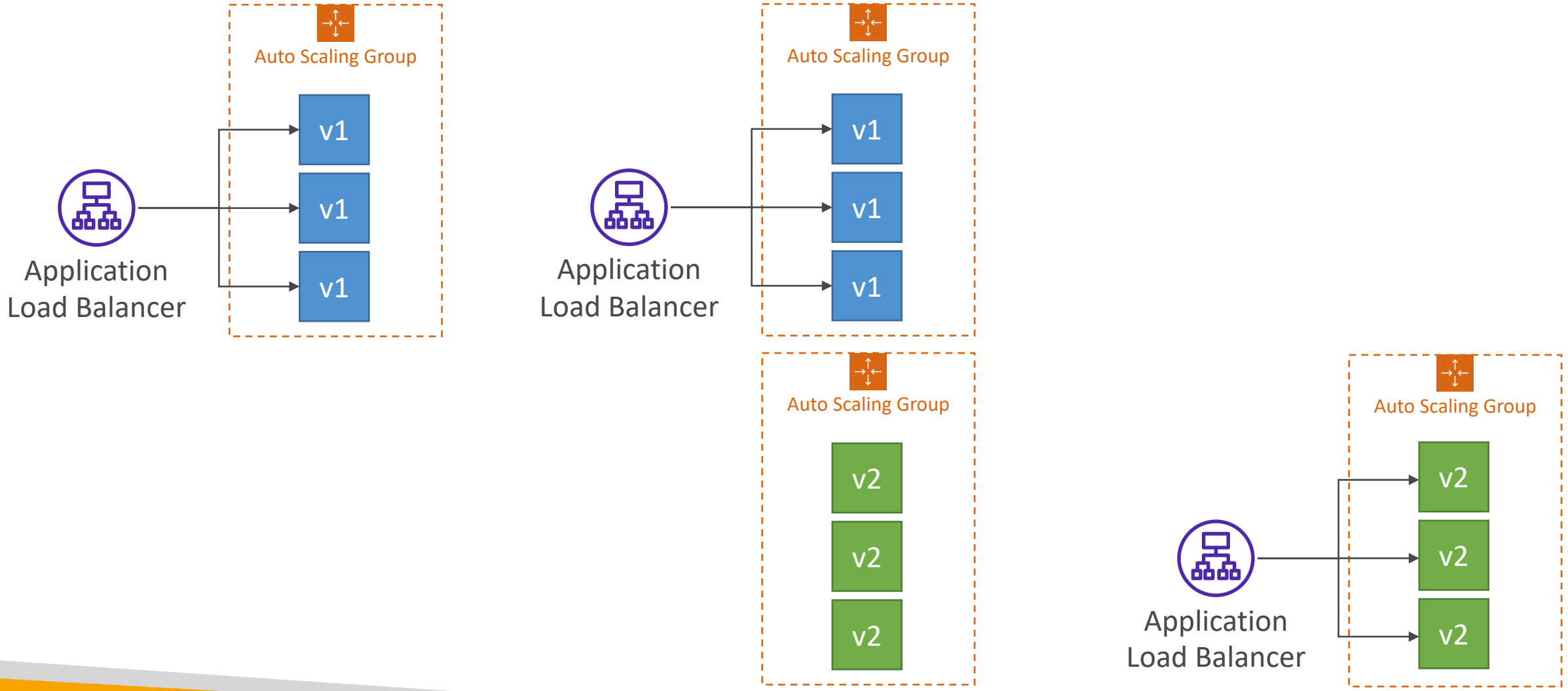
CodeDeploy – EC2/On-premises Platform

- Can deploy to EC2 Instances & on-premises servers
- Perform in-place deployments or blue/green deployments
- Must run the **CodeDeploy Agent** on the target instances
- Define deployment speed
  - AllAtOnce: most downtime
  - HalfAtATime: reduced capacity by 50%
  - OneAtATime: slowest, lowest availability impact
  - Custom: define your %

CodeDeploy – In-Place Deployment

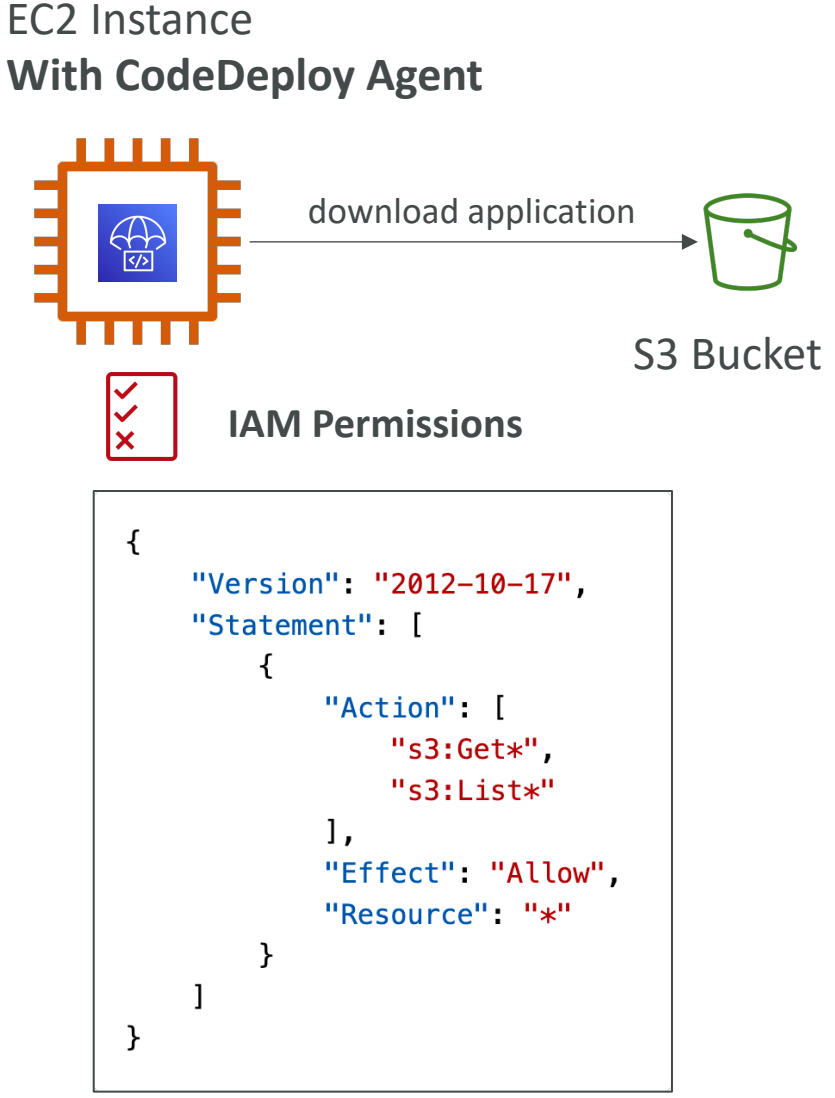


CodeDeploy – Blue-Green Deployment



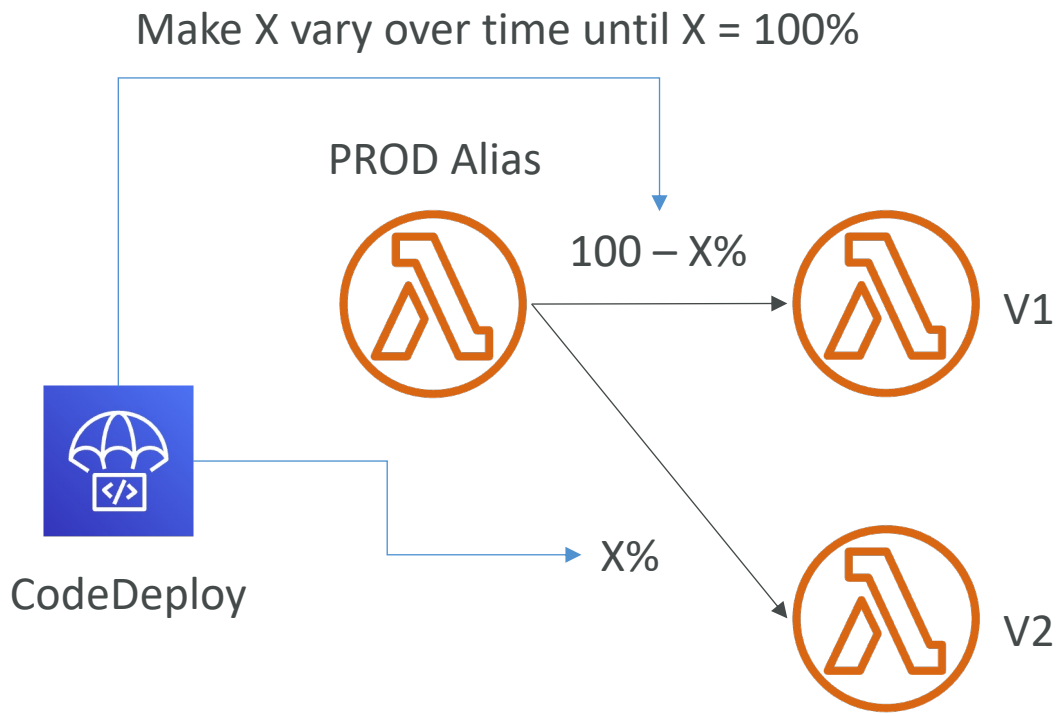
CodeDeploy Agent

- The CodeDeploy Agent must be running on the EC2 instances as a pre-requisites
- It can be installed and updated automatically if you're using Systems Manager
- The EC2 Instances must have sufficient permissions to access Amazon S3 to get deployment bundles



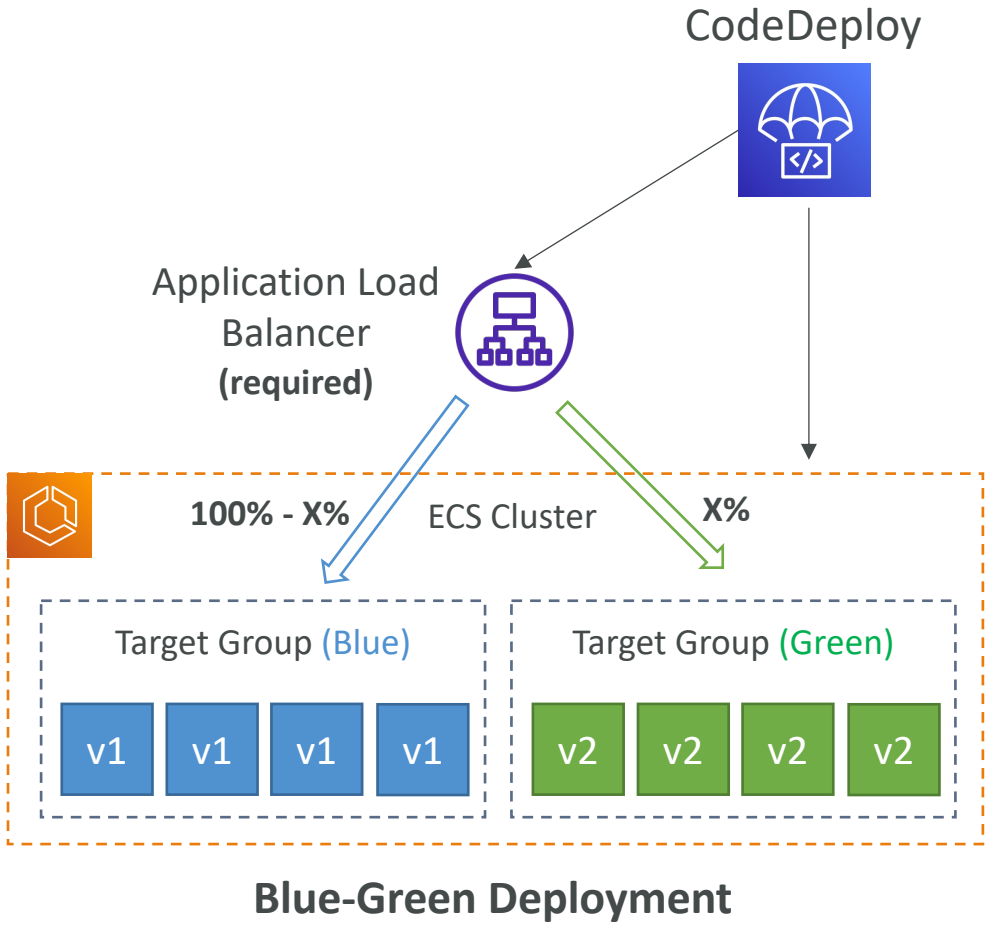
CodeDeploy – Lambda Platform

- **CodeDeploy** can help you automate traffic shift for Lambda aliases
- Feature is integrated within the SAM framework
- **Linear:** grow traffic every N minutes until 100%
  - LambdaLinear10PercentEvery3Minutes
  - LambdaLinear10PercentEvery10Minutes
- **Canary:** try X percent then 100%
  - LambdaCanary10Percent5Minutes
  - LambdaCanary10Percent30Minutes
- **AllAtOnce:** immediate



CodeDeploy – ECS Platform

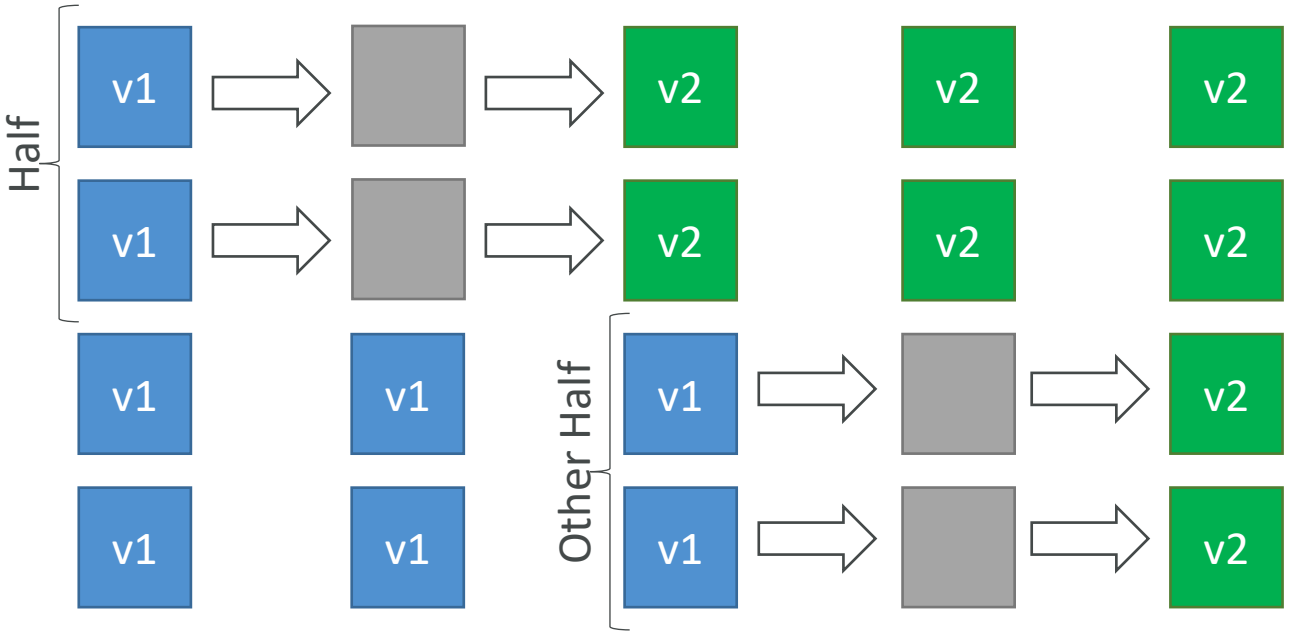
- **CodeDeploy** can help you automate the deployment of a new ECS Task Definition
- Only Blue/Green Deployments
- **Linear:** grow traffic every N minutes until 100%
  - ECSLinear10PercentEvery3Minutes
  - ECSLinear10PercentEvery10Minutes
- **Canary:** try X percent then 100%
  - ECSCanary10Percent5Minutes
  - ECSCanary10Percent30Minutes
- **AllAtOnce:** immediate



Blue-Green Deployment

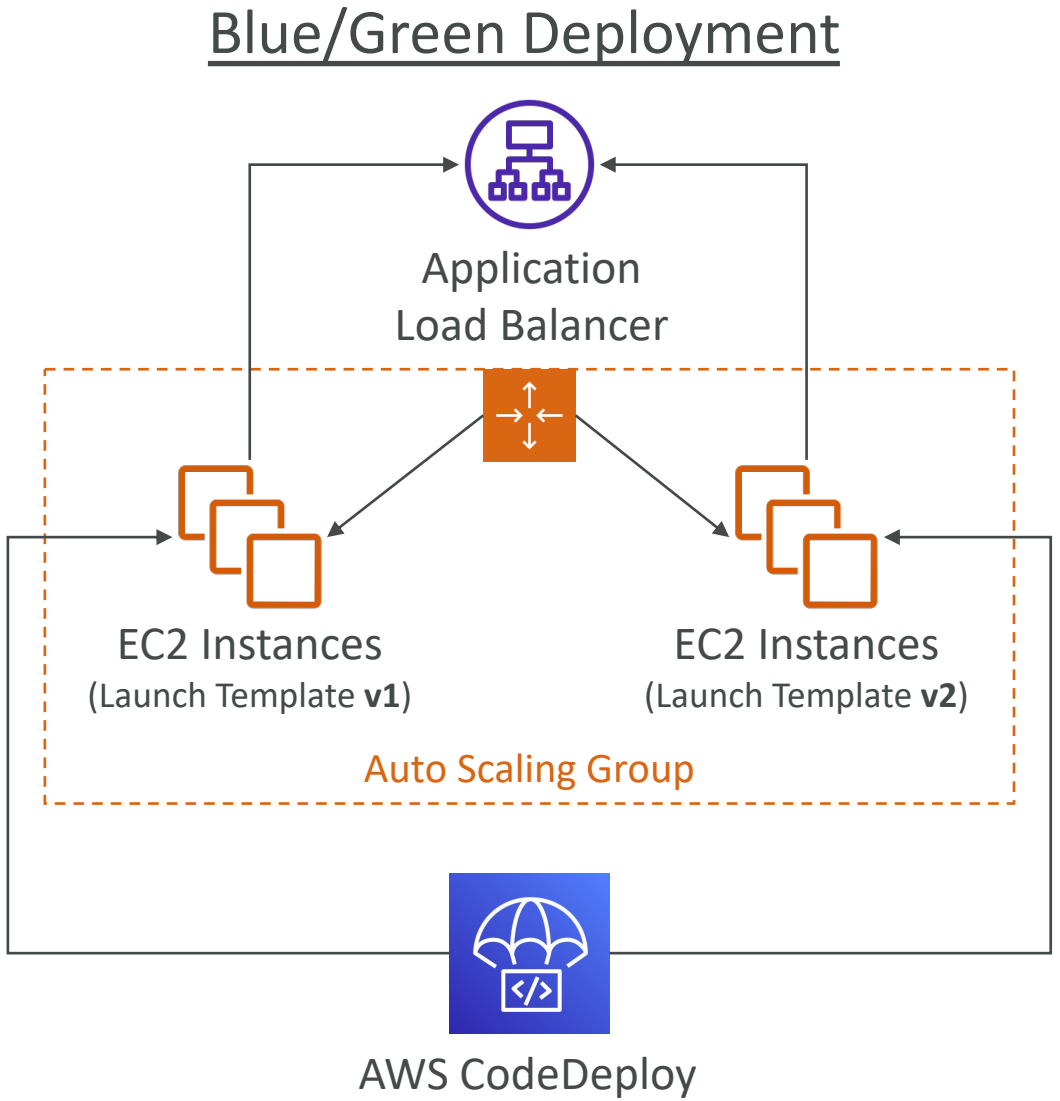
CodeDeploy – Deployment to EC2

- Define how to deploy the application using **appspec.yml** + Deployment Strategy
- Will do In-place update to your fleet of EC2 instances
- Can use hooks to verify the deployment after each deployment phase



CodeDeploy – Deploy to an ASG

- In-place Deployment
  - Updates existing EC2 instances
  - Newly created EC2 instances by an ASG will also get automated deployments
- Blue/Green Deployment
  - A new Auto-Scaling Group is created (settings are copied)
  - Choose how long to keep the old EC2 instances (old ASG)
  - Must be using an ELB



CodeDeploy – Redeploy & Rollbacks

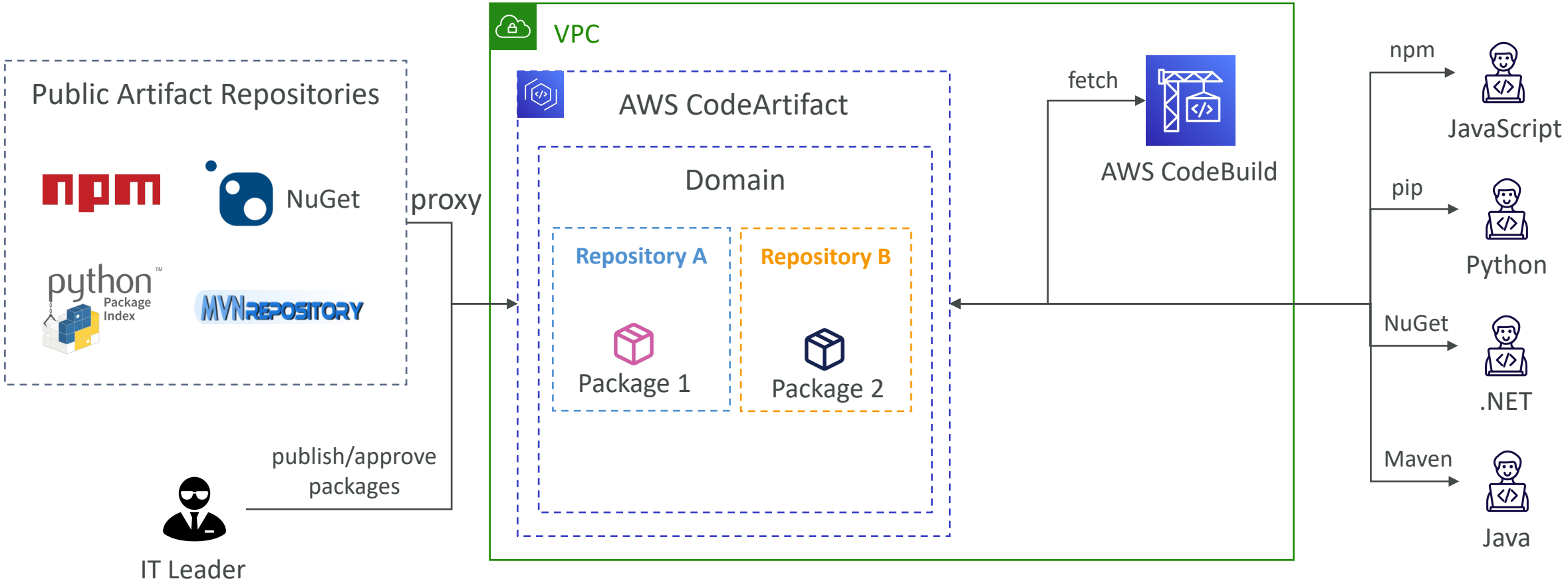
- Rollback = redeploy a previously deployed revision of your application
- Deployments can be rolled back:
  - **Automatically** – rollback when a deployment fails or rollback when a CloudWatch Alarm thresholds are met
  - **Manually**
- Disable Rollbacks — do not perform rollbacks for this deployment

- If a roll back happens, CodeDeploy redeploys the last known good revision as a new deployment (not a restored version)

AWS CodeArtifact

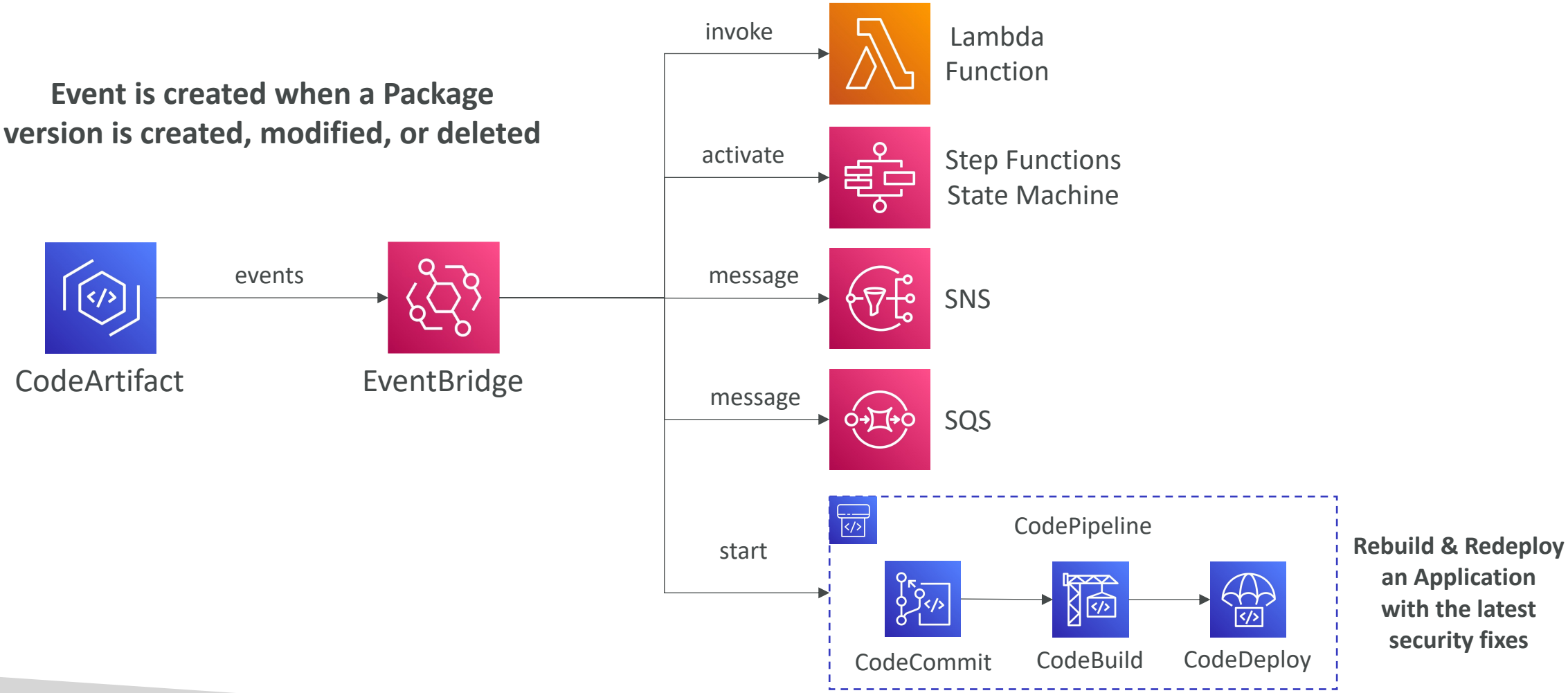
- Software packages depend on each other to be built (also called code dependencies), and new ones are created
- Storing and retrieving these dependencies is called **artifact management**
- Traditionally you need to setup your own artifact management system
- **CodeArtifact** is a secure, scalable, and cost-effective **artifact management** for software development
- Works with common dependency management tools such as Maven, Gradle, npm, yarn, twine, pip, and NuGet
- **Developers and CodeBuild can then retrieve dependencies straight from CodeArtifact**

AWS CodeArtifact



CodeArtifact – EventBridge Integration

Event is created when a Package version is created, modified, or deleted



Rebuild & Redeploy an Application with the latest security fixes

CodeArtifact – Resource Policy

- Can be used to authorize another account to access CodeArtifact
- A given principal can either read all the packages in a repository or none of them


```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "codeartifact:DescribePackageVersion",
                "codeartifact:DescribeRepository",
                "codeartifact:GetPackageVersionReadme",
                "codeartifact:GetRepositoryEndpoint",
                "codeartifact:ListPackages",
                "codeartifact:ListPackageVersions",
                "codeartifact:ListPackageVersionAssets",
                "codeartifact:ListPackageVersionDependencies",
                "codeartifact:ReadFromRepository"
            ],
            "Principal": {
                "AWS": [
                    "arn:aws:iam::123456789012:root",
                    "arn:aws:iam::222333344555:user/bob"
                ]
            },
            "Resource": "*"
        }
    ]
}
```

Repository Resource Policy

AWS Serverless Application Model (SAM)

Taking your Serverless Development to the next level